In [4]:
```
import pandas as pd

# Dataset
marks = [45, 67, 89, 34, 90, 55, 72]
subjects = ['Math', 'Physics', 'Chemistry', 'English', 'CS', 'Biology', 'History
s = pd.Series(marks, index=subjects)
print(" Original Marks (out of 100):\n", s)

# Subjects with marks > 70
print("\n Subjects with marks > 70:\n", s[s > 70])

# Replace marks < 40 with "Fail"
s_updated = s.astype(object)
s_updated[s_updated < 40] = "Fail"
print("\n Marks after replacing <40 with 'Fail':\n", s_updated)

# Count subjects above average
avg = s.mean()
count = (s > avg).sum()
print("\n Average Marks:", avg)
print(" Subjects scoring above average:", count)
```

```
 Original Marks (out of 100):
 Math         45
Physics      67
Chemistry    89
English      34
CS           90
Biology      55
History      72
dtype: int64

 Subjects with marks > 70:
 Chemistry    89
CS           90
History      72
dtype: int64

 Marks after replacing <40 with 'Fail':
 Math           45
Physics        67
Chemistry      89
English      Fail
CS             90
Biology        55
History        72
dtype: object

 Average Marks: 64.57142857142857
 Subjects scoring above average: 4
```

In [8]:
```
import pandas as pd

# Dataset
salary = [25000, 32000, 28000, 40000, 50000]
emp = ['A', 'B', 'C', 'D', 'E']

s = pd.Series(salary, index=emp, dtype=float)  # convert to float to avoid error
print(" Original Salary (₹):\n", s)
```

```
# Increase salary by 10% where salary < 30000
s.loc[s < 30000] = s[s < 30000] * 1.10

# Decrease salary by 5% where salary > 45000
s.loc[s > 45000] = s[s > 45000] * 0.95

print("\n Updated Salary (₹):\n", s)

# Total salary expense
total = s.sum()
print("\n Total Salary Expense (₹):", total)
```

```
 Original Salary (₹):
 A    25000.0
B    32000.0
C    28000.0
D    40000.0
E    50000.0
dtype: float64

 Updated Salary (₹):
 A    27500.0
B    32000.0
C    30800.0
D    40000.0
E    47500.0
dtype: float64

 Total Salary Expense (₹): 177800.0
```

In [9]:

```
import pandas as pd

data = [10, None, 30, None, 50, 60, None]
s = pd.Series(data)
print(" Original Data:\n", s)

# Missing values count
print("\n Missing before:", s.isnull().sum())

# Fill missing with mean
s_filled = s.fillna(s.mean())
print("\n Data after filling missing with mean:\n", s_filled)

# Missing after fill
print("\n Missing after fill:", s_filled.isnull().sum())

# Drop remaining nulls (if any)
s_clean = s_filled.dropna()
print("\n Final cleaned data:\n", s_clean)
```

```
 Original Data:
 0    10.0
1     NaN
2    30.0
3     NaN
4    50.0
5    60.0
6     NaN
dtype: float64

 Missing before: 3

 Data after filling missing with mean:
 0    10.0
1    37.5
2    30.0
3    37.5
4    50.0
5    60.0
6    37.5
dtype: float64

 Missing after fill: 0

 Final cleaned data:
 0    10.0
1    37.5
2    30.0
3    37.5
4    50.0
5    60.0
6    37.5
dtype: float64
```

In [10]:

```python
import pandas as pd

marks = [95, 82, 67, 58, 73, 49, 88]
students = ['S1','S2','S3','S4','S5','S6','S7']
s = pd.Series(marks, index=students)
print(" Student Marks:\n", s)

# Assign grades
def assign_grade(x):
    if x >= 90: return 'A'
    elif x >= 75: return 'B'
    elif x >= 60: return 'C'
    else: return 'D'

grades = s.apply(assign_grade)
print("\n Grades Assigned:\n", grades)
print("\n Grade Count:\n", grades.value_counts())
```

```
 Student Marks:
 S1    95
S2    82
S3    67
S4    58
S5    73
S6    49
S7    88
dtype: int64

 Grades Assigned:
 S1    A
S2    B
S3    C
S4    D
S5    C
S6    D
S7    B
dtype: str

 Grade Count:
 B    2
C    2
D    2
A    1
Name: count, dtype: int64
```

In [11]:

```python
import pandas as pd

s1 = pd.Series([10, 20, 30], index=['A','B','C'])
s2 = pd.Series([5, 15, 25], index=['B','C','D'])
print(" Series 1:\n", s1)
print("\n Series 2:\n", s2)

# Add series (NaN appears)
result = s1 + s2
print("\n Result (with NaN):\n", result)

# Replace NaN with 0
result_clean = result.fillna(0)
print("\n Result after replacing NaN with 0:\n", result_clean)
```

```
 Series 1:
 A    10
B    20
C    30
dtype: int64

 Series 2:
 B     5
C    15
D    25
dtype: int64

 Result (with NaN):
 A     NaN
B    25.0
C    45.0
D     NaN
dtype: float64

 Result after replacing NaN with 0:
 A     0.0
B    25.0
C    45.0
D     0.0
dtype: float64
```

In [12]:

```python
import pandas as pd

scores = [88, 92, 79, 93, 85]
players = ['P1','P2','P3','P4','P5']
s = pd.Series(scores, index=players)
print("  Player Scores:\n", s)

# Ranking (highest score = rank 1)
rank = s.rank(ascending=False)
print("\n  Player Ranking:\n", rank)

# Top 3 players
top3 = s.sort_values(ascending=False).head(3)
print("\n  Top 3 Players:\n", top3)

# Lowest score player
print("\n  Player with lowest score:", s.idxmin())
```

```
  Player Scores:
 P1    88
P2    92
P3    79
P4    93
P5    85
dtype: int64

  Player Ranking:
 P1    3.0
P2    2.0
P3    5.0
P4    1.0
P5    4.0
dtype: float64

  Top 3 Players:
 P4    93
P2    92
P1    88
dtype: int64

  Player with lowest score: P3
```

In [13]:

```python
import pandas as pd

names = ['saloni', 'ujjwal', 'sneha', 'kiran', 'amit']
s = pd.Series(names)
print("  Original Names:\n", s)

# Uppercase
print("\n  Uppercase Names:\n", s.str.upper())

# Names containing 'a'
print("\n  Names containing 'a':\n", s[s.str.contains('a')])

# Replace 'anuj' with 'Anuj Kumar'
s_updated = s.replace('anuj','Anuj Kumar')
print("\n  Updated Names:\n", s_updated)
```

```
  Original Names:
 0    saloni
1    ujjwal
2     sneha
3     kiran
4      amit
dtype: str

  Uppercase Names:
 0    SALONI
1    UJJWAL
2     SNEHA
3     KIRAN
4      AMIT
dtype: str

  Names containing 'a':
 0    saloni
1    ujjwal
2     sneha
3     kiran
4      amit
dtype: str

  Updated Names:
 0    saloni
1    ujjwal
2     sneha
3     kiran
4      amit
dtype: str
```

In [14]:

```python
import pandas as pd

data = [5, None, 15, 25, None, 35, 45]
s = pd.Series(data)
print("  Original Data:\n", s)

# Fill missing → filter >20 → sort descending → reset index
result = (s.fillna(s.median())
            [lambda x: x>20]
            .sort_values(ascending=False)
            .reset_index(drop=True))
print("\n  Final Result after chained operations:\n", result)
```

```
  Original Data:
 0     5.0
1     NaN
2    15.0
3    25.0
4     NaN
5    35.0
6    45.0
dtype: float64

  Final Result after chained operations:
 0    45.0
1    35.0
2    25.0
3    25.0
4    25.0
dtype: float64
```

In [15]:

```python
import pandas as pd

sales = [200, 450, 300, 150, 500, 700, 100]
days = ['Mon','Tue','Wed','Thu','Fri','Sat','Sun']
s = pd.Series(sales, index=days)
print("  Weekly Sales (units):\n", s)

# Best and worst day
print("\n  Best Sales Day:", s.idxmax())
print("  Worst Sales Day:", s.idxmin())

# Total & average sales
print("\n  Total Sales:", s.sum())
print("  Average Sales:", s.mean())

# Apply 20% bonus for sales > 400
bonus = s.copy()
bonus[bonus>400] *= 1.20
print("\n  Sales after 20% bonus:\n", bonus)

# Sort descending
print("\n  Sales sorted descending:\n", s.sort_values(ascending=False))
```

```
  Weekly Sales (units):
 Mon    200
Tue    450
Wed    300
Thu    150
Fri    500
Sat    700
Sun    100
dtype: int64

  Best Sales Day: Sat
  Worst Sales Day: Sun

  Total Sales: 2400
  Average Sales: 342.85714285714283

  Sales after 20% bonus:
 Mon    200
Tue    540
Wed    300
Thu    150
Fri    600
Sat    840
Sun    100
dtype: int64

  Sales sorted descending:
 Sat    700
Fri    500
Tue    450
Wed    300
Mon    200
Thu    150
Sun    100
dtype: int64
```

In [ ]: